

Sriteja Kataru — AIE7 Certification Challenge Write-up

1. Write a succinct 1-sentence description of the problem

The vast majority of university students lack quick access to health information that combines medical accuracy with a student-specific focus, forcing them to choose between unvetted, quick answers from flagship AI chatbots/social media and time-consuming research.

2. Write 1-2 paragraphs on why this is a problem for your specific user.

When students experience health concerns or simply need wellness-related advice, from the typical exam stress to wondering what to eat before class to maximize focus, they need immediate & trustworthy guidance tailored to their unique situation. General AI chatbots like ChatGPT, while they are convenient, pull from broad internet sources without distinguishing between peer-reviewed medical research and wellness blogs promoting pseudoscience. Not to mention, the amount of health-related misinformation online is becoming rampant. Also, these AI tools lack the contextual understanding that a 20-year-old living in a dorm has vastly different health needs and constraints than a middle-aged adult.

The alternative, manually researching across university health portals, PubMed databases, and verified news sources, demands time that students juggling classes, work, and social lives simply don't have. This dilemma drives them to the most dangerous option: social media and forums where algorithms are primed to display more misinformation than facts. A student searching "constant exhaustion" might find anything from helpful sleep hygiene tips to dangerous supplement recommendations, with no way to quickly verify what's safe and relevant to their actual situation. The result is delayed medical care (if needed), unnecessary anxiety, and potentially harmful or useless self-treatment.

3. Write 1-2 paragraphs on your proposed solution. How will it look and feel to the user?

The Student Health Advisor is an interactive AI tool that provides students with instant and evidence-based health guidance through a simple yet friendly chat interface that blends four critical sources of information. The AI agent first searches through built-in university health guides covering nutrition, stress, sleep, and mental health, while also pulling from current health news, peer-reviewed medical research, and general arXiv papers to output one comprehensive and relevant response. This approach makes sure that students get not only credible answers but information spanning multiple types of sources.

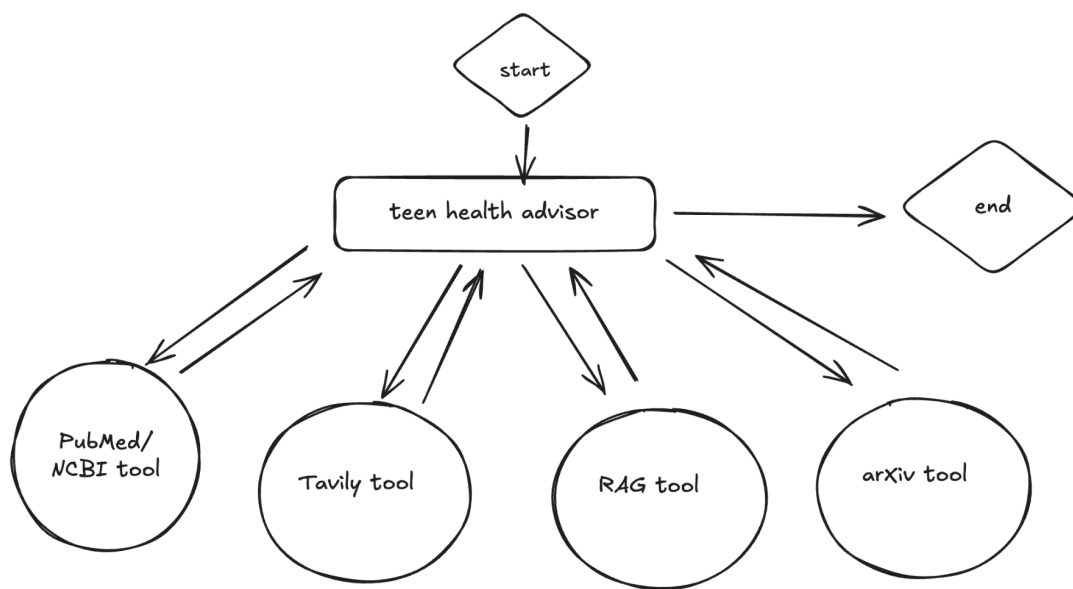
The tool's core strength is its multi-source synthesis of information that would typically take hours to gather manually. Let's say a student asks about sleep problems—rather than receiving generic advice, they would receive guidance blending dorm-specific sleep hygiene tips from an Ivy League-curated health guide with recent research on how all-nighters hinder academic performance with current articles about sleep apps popular among college students. The simple UI makes the experience friendlier and accessible. By completely erasing the need to search

multiple databases or verify source credibility, the tool gives time-pressed students immediate access to trustworthy health information when they need it most.

4. Describe the tools you plan to use in each part of your stack. Write one sentence on why you made each tooling choice.
 - a. LLM
 - i. For the RAG Model: GPT 4.1 Nano — this model is incredibly cheap and works well with simple tasks (in this case, the task would be to take the retrieved chunks, make them into coherent answers, and sometimes format with citations)
 - ii. Models: GPT 4o mini — Based on advice I received from my peer supporter Can, this agent should have an LLM with good reasoning capabilities so it can make the best tooling decisions. GPT 4o would be the best model here, but 4o mini gives you 90% of GPT-4o's capability at 10% of the cost.
 - b. Embedding Model
 - i. Text-embedding-3-small — based on what we learned in the sessions, this embedding model performs reliably well, and I found online (<https://www.datacamp.com/tutorial/exploring-text-embedding-3-large-new-openai-embeddings>) that this embedding model has the right balance of performance and cost for semantic search across these health PDFs.
 - c. Orchestration
 - i. LangGraph/LangChain — this is the framework that we've been using the most in the sessions, but that set aside, it allows the agent control flow to dynamically select between the four tools (RAG, Tavily, PubMed, arXiv) a non-constant amount of times.
 - d. Vector Database
 - i. Qdrant — again, this is the only vector database we've used so far, and based on my research, this database seems to consistently perform well (relative to the other flagship ones).
 - e. Monitoring
 - i. LangSmith — seems like the natural choice because it provides LangChain/LangGraph-native observability to track the agent's decisions and tool calls, and we can see the full conversation flows for debugging/latency distributions.
 - f. Evaluation
 - i. RAGAS — this is one of the best systems for evaluating RAG systems. We can select metrics like answer relevance, faithfulness, and context precision in a fairly simple manner. I am also most comfortable with RAGAS.
 - g. User Interface
 - i. Next.js - React framework for the web interface. Provides smooth streaming responses and has simple built-in chat UI components.

- ii. TypeScript - For type safety (tsconfig.json present)
 - iii. Tailwind CSS - For styling (postcss.config.mjs suggests Tailwind)
 - iv. ESLint - For code linting
- h. (Optional) Serving & Inference
- 5. Where will you use an agent or agents? What will you use “agentic reasoning” for in your app?
 - a. I use a single agent (implemented with LangGraph) as the central decision-maker that analyzes each student health query and determines which tools to use from a set of 4 tools—whether to search the provided university health guide pdfs for general wellness info, query PubMed for medical-related evidence, analyze arXiv papers for general scientific research, or search current news via Tavily for recent health trends. The agent is encouraged to first search the university health guides (RAG tool), but is also prompted to utilize multiple tools if the query is complex and the answer would be enhanced with multiple tool calls.

Here is a flowchart of the app logic:



- 6. Describe all of your data sources and external APIs, and describe what you'll use them for.

Data Sources

- PDF files of the university student health and wellness resources
- Some guides come from schools like Columbia University, Penn State, University of Colorado at Boulder, and more.
- Made sure to include one PDF for each essential pillar of teen health: nutrition, sleep, stress, exercise, and mental health.

- File names:
 - Accessible-online-version-Manage-Stress-Workbook-PennState-final-2i5sewu.pdf
 - CollegeStudentMentalHealthActionToolkit.pdf
 - Guide-to-college-student-nutrition-survival-flccc.pdf
 - SHS_PatientEd_SleepHygiene.pdf
 - Sleep_Healthy Habits.pdf
 - Sleep-LetsCU.pdf
 - Workouts for College Students.pdf

External APIs

1. OpenAI API — for the LLM responses and text embeddings (GPT 4o mini, GPT 4.1 nano, text-embedding-3-small, GPT 4o for token counting and text splitting optimization)
2. Tavily search API — web search for current health information/latest news
3. Cohere API — document reranking for better retrieval (contextual compression). I use CohereRerank (rerank-v3.5 model)
4. arXiv API — general STEM research paper search
5. Pubmed/NCBI (biopython) API — medical research paper search
6. LangChain/LangSmith API — orchestration, LangChain tracing and monitoring

Some other external imports for the data processing pipeline:

- Qdrant: In-memory vector store for document embeddings
- tiktoken: Token counting and text splitting
- PyMuPDF: PDF document parsing
- RecursiveCharacterTextSplitter: Document chunking

7. Describe the default chunking strategy that you will use. Why did you make this decision?
 - a. I'm using RecursiveCharacterTextSplitter with 750-character chunks and no overlap because it's the industry standard that works reliably across diverse document types—particularly important for health PDFs that could have a variety of text arrangements from bullet points to paragraphs. The chunk size also hits the sweet spot for health information, because it is large enough to capture complete medical recommendations and context while small enough to maintain high relevance scores during the retrieval process. Semantic chunking, which is also appealing, adds unnecessary complexity and often performs worse on structured documents like university health guides where the existing formatting already provides natural semantic boundaries.
8. [Optional] Will you need specific data for any other part of your application? If so, explain.
 - a. n/a
9. Build an end-to-end prototype and deploy it to a *local* endpoint
 - a. done

10. Assess your pipeline using the RAGAS framework including key metrics faithfulness, response relevance, context precision, and context recall. Provide a table of your output results.

a. Done

```
{'faithfulness': 0.9071,  
  
'factual_correctness(mode=f1)': 0.5044,  
  
'answer_relevancy': 0.9706,  
  
'context_recall': 0.9352,  
  
'context_entity_recall': 0.2072,  
  
'noise_sensitivity(mode=relevant)': 0.4790,  
  
'context_precision': 1.0000}
```

11. What conclusions can you draw about the performance and effectiveness of your pipeline with this information?

- a. Faithfulness: Got 90.7%, which means the system generates information that, for the most part, stays true to the context. 90% is high, but considering that we are working with health-related advice, I feel that the number should be upwards of 97% for credibility purposes.
- b. Factual Correctness: Only 50%, which means it is essentially a coin flip on whether facts are accurate, though this might be because the test expects similar/identical wording.
- c. Answer Relevancy: Scored 97%, so responses directly relate to what was asked.
- d. Context Recall: 93.5%—most of the time, the system retrieves almost, if not all, the documents relevant to the query.
- e. Context Entity Recall: Worst score at 20.7%—this metric suggests that our RAG chain is not good at pulling out specific details like numbers, dates, or medical terms that students need. But this number is pretty much in line with context entity recalls based on previous assignments.
- f. Noise Sensitivity: Only maintained 47.1% performance when irrelevant info was mixed in, which means that our pipeline gets easily distracted by unrelated content.
- g. Context Precision: Got 100% all retrieved chunks are relevant and actually about what the student asked.

12. Describe the retrieval techniques that you plan to try and to assess in your application.

Write one sentence on why you believe each technique will be useful for your use case.

- a. Ensemble Retrieval — combines semantic search with multi-query retrieval, parent document retrieval, and contextual compression. Essentially provides the

best of all worlds. I would think having this comprehensiveness is important because health queries can be quite diverse.

- b. Multi-Query Retrieval — particularly important because students often ask health questions in vague or colloquial terms (e.g., "I feel weird after pulling all-nighters"), and generating many similar queries from the user's query will help capture relevant information from formal medical PDFs that might use different terminology like "sleep deprivation effects" or "circadian rhythm disruption."
 - c. Parent-Document Retrieval — since health guidance often requires full context to be actionable and safe (you can't just extract a single piece of advice without the surrounding warnings and conditions), this method makes sure that students receive complete sections from health guides rather than potentially misleading fragments or pieces of information that don't make sense when they are isolated.
 - d. Contextual Compression (Rerank) — given that the agent searches through a rich database, reranking chunks could be important for re-prioritizing the most relevant information.
13. Test a host of advanced retrieval techniques on your application.
- a. done
14. How does the performance compare to your original RAG application? Test the fine-tuned embedding model using the RAGAS frameworks to quantify any improvements. Provide results in a table.

Here is the table of the results:

SIDE-BY-SIDE RETRIEVER COMPARISON						
Metric	NAIVE	MULTI_QUERY	PARENT_DOCUMENT	COMPRESSION	ENSEMBLE	
Avg Latency (s)	3.47880	5.137500	2.011800	3.126700	6.810600	
Avg Cost (\$)	0.00037	0.000361	0.000338	0.000303	0.000605	
Faithfulness	0.97400	0.912700	0.821700	0.938900	0.986100	
Factual Correctness	0.54560	0.537800	0.553300	0.607800	0.483300	
Answer Relevancy	0.96580	0.964300	0.858700	0.960300	0.975200	
Context Recall	0.95000	0.972200	0.972200	0.935200	0.972200	
Context Entity Recall	0.20670	0.123400	0.218800	0.169700	0.187700	
Noise Sensitivity	0.39530	0.369800	0.284000	0.289400	0.495700	
Context Precision	1.00000	0.861100	0.944400	0.944400	0.801500	

Here are some observations:

- Compared to the naive retriever (original RAG application), almost all the advanced retrievers had higher latencies—only the parent document and compression retrievers were faster (2.01s and 3.13s vs 3.48s). Ensemble is evidently really slow at 6.81s since it's running multiple retrieval methods at the same time.
- As for cost, they're all pretty much identical (around \$0.0003 per query) except ensemble which costs almost double at \$0.0006, probably because it's using multiple methods together.

- The original RAG retriever scored second-highest in faithfulness (97.4%), which means it rarely adds information that wasn't in the retrieved documents. Only the ensemble retriever beat it slightly at 98.6%.
- For factual correctness, the contextual compression retriever performed best (60.8%) while naive came in third (54.6%). Overall, advanced retrievers are somewhat better at extracting specific facts from the documents but the naive retriever is still at the 50th percentile.
- Most of the answer relevancy scores were right around the same (~96%), except the parent document retriever dropped to 85.9%.
- Context recall was also pretty consistent (93-97%), which means that all retrievers find relevant documents well.
- Context entity recall was poor across the board (12-22%). All of the methods struggle with pulling out specific details like names or numbers, which is a bad thing for a health-related application. But the naive retriever still got second-highest in this criteria.
- The naive retriever achieved perfect context precision (100%), meaning every chunk it retrieved was relevant. The others were lower (80-94%), so they pulled in some irrelevant content.

I would say that, interestingly, the naive retriever remains really competitive. It has perfect precision, strong faithfulness, and solid performance across metrics while being reasonably fast. The advanced methods only show some small improvements in specific areas but add complexity and latency. I would still go with naive retrieval for my app.

15. Articulate the changes that you expect to make to your app in the second half of the course. How will you improve your application?
 - a. Explain which source came from where.
 - i. Currently, outputs could contain lines that say “according to a few reputable health blogs” or semi-specific information regarding the sources that it used to generate the responses, However, considering we have 4 different tools and the ability to search the web/research papers, it would be helpful for the user if the LLM explicitly states which exact sources the agent took information from. Sometimes the agent provides links when using the Tavily search tool and lists the article & author names when using the academic research tools but this should be a built-in capability that happens every time.
 - b. Persisted memory
 - i. Currently, you can communicate with the agent on a singular query-to-response basis, but the agent has no memory from message to message. It would be nice to implement memory between messages and sessions for an even more personalized experience.
 - c. Take the output and then do something (calendar, make a pdf)
 - i. Currently, this tool is only a chatbot that evidently outputs a chat message. However, I would like this tool to have more capabilities which are geared towards users taking the next step and making definite changes with their health. The text output given from the agent could then

automatically lead to something else, such as a calendar invite to track a habit or the creation of a personalized PDF containing vital nutrition information so the user can hang on to that information for later.